

Trabalho de Implementação 1 CIC0201

Cifra de Vigenère: Implementação e Criptoanálise

Mateus Valério*, Maylla Krislainy†

Departamento de Ciência da Computação, Universidade de Brasília

Matrícula: *19/0035616 †19/0043873

Resumo—Este trabalho apresenta a implementação da cifra de Vigenère, incluindo cifração, decifração e um ataque de criptoanálise para recuperação da chave a partir do criptograma. O ataque combina o Exame de Kasiski, o Índice de Coincidência (Teste de Friedman) e análise de frequência de letras, aplicados a criptogramas em português e em inglês. Discutem-se também os limites teóricos e práticos do ataque, incluindo a dependência do volume de texto disponível para a confiabilidade da estimativa do tamanho da chave.

Index Terms—cifra de Vigenère, criptoanálise, índice de coincidência, análise de frequência

I. INTRODUÇÃO

A cifra de Vigenère é um esquema de substituição que estende a Cifra de César ao aplicar deslocamentos distintos a cada posição do plaintext, determinados ciclicamente por uma palavra-chave. Historicamente considerada inquebrável por séculos, sua vulnerabilidade foi demonstrada por métodos estatísticos capazes de estimar o tamanho da chave e, a partir disso, reduzir o problema a múltiplas instâncias independentes da Cifra de César. Este trabalho tem dois objetivos: (i) implementar a cifração e decifração da cifra de Vigenère; e (ii) implementar um ataque de criptoanálise capaz de recuperar a chave a partir de um criptograma, sem conhecimento prévio de seu tamanho ou conteúdo. O código-fonte completo e os scripts de teste desenvolvidos estão disponíveis publicamente em: <https://github.com/im-maV/unb-security-comp.git>.

II. FUNDAMENTAÇÃO TEÓRICA

A. Domínio e Normalização

O domínio utilizado é o alfabeto de A a Z (equivalentemente, a-z), independentemente do uso de maiúsculas ou minúsculas no texto original: ambas são cifradas da mesma forma. O algoritmo verifica se um caractere é maiúsculo e, em caso positivo, converte-o para minúsculo, aplica o deslocamento (*shift*) e reverte o caractere para maiúsculo ao final. Por exemplo, com a chave KEY = ABC e o plaintext O1a, obtém-se o criptograma Omc.

Espaços e demais caracteres especiais não são cifrados nesta implementação. Números também são mantidos sem alteração. Uma alternativa possível seria normalizar caracteres acentuados para sua forma sem acento (ex.: á → a, ç → c) antes da cifração; essa abordagem, no entanto, introduz o problema de não ser possível restaurar os acentos originais na decifração.

B. Cifração e Decifração

Sejam p_i o valor numérico da i -ésima letra do plaintext, k_i o valor numérico da letra correspondente da chave (repetida ciclicamente) e c_i o valor numérico da letra cifrada correspondente. A cifração e a decifração são definidas por:

$$c_i = (p_i + k_i) \bmod 26 \quad (1)$$

$$p_i = (c_i - k_i) \bmod 26 \quad (2)$$

C. Segurança Perfeita e Quase Perfeita

Caso o tamanho da chave seja maior ou igual ao tamanho do plaintext, e ambos sejam suficientemente grandes (por exemplo, maior que 14 caracteres), e os caracteres da chave sejam estritamente aleatórios, qualquer ataque por força bruta ou por análise estatística torna-se inviável. Isso ocorre porque o atacante não tem como estimar o tamanho da chave a priori, já que não há repetições que permitam essa estimativa. Além disso, para cada tamanho de chave possível, os valores de Índice de Coincidência (IC) resultantes tendem a ficar próximos de $1/26$, valor bem abaixo do IC esperado para linguagem natural.

Essa propriedade é conhecida como **Segredo Perfeito** (*Perfect Secrecy*), formalizada por C. E. Shannon em 1949. O sistema que a atinge é o **One-Time Pad**: Shannon demonstra que, se a chave é verdadeiramente aleatória, uniformemente distribuída, usada uma única vez e possui o mesmo tamanho da mensagem, então todo plaintext do mesmo tamanho é igualmente provável dado o criptograma observado [1].

D. Garantias de Segurança

1) **Ataque de Força Bruta**: Para uma chave de m caracteres, cada um podendo assumir 26 valores, o espaço de busca é 26^m :

m	Chaves possíveis
1	26
2	676
⋮	⋮
8	$2,09 \times 10^{11}$
14	$6,45 \times 10^{19}$

Quando o plaintext é pequeno, o tamanho da chave torna-se menos relevante para a viabilidade do ataque, mas surge um novo problema: um mesmo criptograma pode corresponder a múltiplos plaintext plausíveis, tornando impossível determinar

qual é o correto. Por exemplo, o criptograma QXZ pode ser decifrado tanto para THE (chave XQV) quanto para CAT (chave OXG). No total, existem $26^3 = 17.576$ plaintext possíveis para um criptograma de 3 letras; se a língua do texto for inglês, apenas uma fração pequena (aproximadamente mil) corresponderia a palavras válidas.

2) **Ataque Estatístico:** Para chaves suficientemente grandes e textos relativamente longos (por exemplo, chave de tamanho 14), a enumeração por força bruta torna-se computacionalmente inviável. Nesse cenário, a cifra passa a ser vulnerável a ataques estatísticos: é possível estimar o tamanho da chave e, consequentemente, os deslocamentos aplicados, por meio do Exame de Kasiski [2], [3] e do Índice de Coincidência (Teste de Friedman) [4], [5], calculado para cada subalfabeto. Quanto menor a chave em relação ao texto, mais fácil se torna esse tipo de ataque.

Já para plaintext pequenos, o ataque estatístico se torna difícil, não por limitação computacional, mas pela escassez de informação estatística disponível, o que reduz a confiabilidade do resultado. Nesses casos, a força bruta tende a ser a alternativa mais indicada, ainda que, como discutido, também limitada pela ambiguidade entre múltiplos plaintext plausíveis quando o texto é curto.

E. Modelo de Ataque Utilizado

- 1) **Estimativa do tamanho da chave:** em vez de duas checagens independentes, o Exame de Kasiski [2], [3] atua como primeira etapa, gerando uma lista ordenada de tamanhos candidatos a partir dos divisores mais frequentes das distâncias entre sequências repetidas no criptograma. Em seguida, para cada candidato dessa lista, calcula-se o Índice de Coincidência médio dos subconjuntos correspondentes (Teste de Friedman) [4], [5]; o candidato com maior IC médio é escolhido como tamanho final da chave. Essa combinação reduz o espaço de busca do Friedman ao conjunto de candidatos já sugeridos pelo Kasiski. Formalmente, essa etapa consiste em dividir o criptograma em k colunas, de modo que a coluna j (com $0 \leq j < k$) contenha apenas os caracteres cujo índice i no criptograma satisfaz $i \equiv j \pmod{k}$. Como todos os caracteres de uma mesma coluna foram deslocados pelo mesmo caractere de chave k_j , cada coluna passa a se comportar isoladamente como uma Cifra de César. Para cada coluna, calcula-se o Índice de Coincidência [4], [5]:

$$IC = \frac{1}{n(n-1)} \sum_{i=1}^{26} f_i(f_i - 1) \quad (3)$$

em que f_i é a quantidade de ocorrências da i -ésima letra do alfabeto na coluna e n é o número de caracteres da coluna (Equação 3, implementada em `calculate_ic`). Friedman demonstrou que, para uma distribuição uniforme de letras – como a que se aproximaria de um texto polialfabético ideal, em que cada posição mistura vários alfabetos de César – o IC tende

a 0,038; já para uma distribuição não uniforme, como a do idioma inglês, o IC tende a 0,065, valor que reflete a frequência natural das letras da língua. Assim, para cada candidato k sugerido pelo Exame de Kasiski, calcula-se o IC médio de suas k colunas,

$$\overline{IC}(k) = \frac{1}{k} \sum_{j=0}^{k-1} IC_j \quad (4)$$

e o candidato cujo $\overline{IC}(k)$ mais se aproxima do valor característico do idioma do texto claro é selecionado como o tamanho estimado da chave, conforme implementado em `find_key_length_from_columns`.

- 2) **Separação em subgrupos:** o criptograma é dividido em grupos de acordo com o tamanho de chave estimado, de forma que cada grupo contenha apenas caracteres cifrados com o mesmo k_i , reduzindo cada grupo a uma Cifra de César independente.
- 3) **Análise de frequência:** realizada sobre cada grupo de César para estimar as probabilidades de ocorrência de cada letra e, a partir disso, descobrir o k_i utilizado naquele grupo, por meio de correlação entre a distribuição observada e a distribuição esperada da língua [6] (Índice de Coincidência Mútuo). Mais precisamente, para cada coluna correspondente ao tamanho de chave estimado, determina-se o deslocamento k_j efetivamente empregado testando-se as 26 hipóteses possíveis. Sejam q_i a frequência relativa observada da i -ésima letra do alfabeto na coluna (isto é, $q_i = f_i/n$) e p_i a frequência esperada dessa mesma letra no idioma do texto claro [6]. Para cada deslocamento candidato $s \in \{0, 1, \dots, 25\}$, calcula-se o Índice de Coincidência Mútuo:

$$M(s) = \sum_{i=0}^{25} p_i \cdot q_{(i+s) \bmod 26} \quad (5)$$

Quando s coincide com o deslocamento real k_j , a distribuição observada q corresponde, em teoria, à própria distribuição p apenas realinhada, de modo que $M(k_j) \approx \sum_{i=0}^{25} p_i^2$ – valor que se aproxima do IC característico do idioma (por exemplo, $\approx 0,065$ para o inglês). Dessa forma, o deslocamento s que maximiza $M(s)$ é tomado como o caractere k_j da chave naquela posição (Equação 5, implementada em `analyze_column_shift`). Repetindo-se esse procedimento para todas as k colunas e concatenando-se os deslocamentos encontrados, obtém-se a chave candidata completa, conforme realizado em `reconstruct_key_from_columns`.

- 4) **Recuperação da chave e decifração:** com o tamanho e o conteúdo da chave estimados, o criptograma é decifrado para obter o plaintext candidato.

III. AMBIENTE EXPERIMENTAL E ANÁLISE DE RESULTADOS

A. Descrição do Cenário

A implementação foi feita em Python 3, utilizando apenas bibliotecas da biblioteca padrão (`collections`, `itertools`, `pathlib`, `argparse`), sem dependências externas. O código está organizado em quatro módulos principais:

- `módulo vigenere`: implementa a cifração e a decifração propriamente ditas, além de suporte a leitura e escrita de arquivos em blocos (*streaming*), permitindo processar arquivos maiores que a memória disponível sem carregá-los por completo;
- `frequencies.py`: contém as tabelas de frequência de letras para português e inglês, bem como o Índice de Coincidência teórico esperado para cada idioma, usado como referência de comparação;
- `kasiski_test.py`: implementa o Exame de Kasiski, localizando sequências repetidas no criptograma e calculando os divisores mais votados entre as distâncias observadas;
- `módulo break_the_rules`: orquestra o ataque completo: estimativa do tamanho da chave (Kasiski seguido de Friedman/IC), separação em subgrupos, análise de frequência por subgrupo e decifração final. Suporta três modos de entrada: texto informado diretamente, arquivo em disco (processado em blocos) e casos de teste pré-definidos com gabarito conhecido, usados para validação.

Os casos de teste incluem criptogramas em português e em inglês, cifrados com chaves de tamanhos variados (10, 14 e 20 caracteres), permitindo validar o ataque em diferentes condições de idioma e comprimento de chave.

B. Análise de Resultados

O ataque foi validado com sucesso nos criptogramas de maior volume, em ambos os idiomas: para os casos com aproximadamente 800 a 1000 letras e chave de 20 caracteres, o Exame de Kasiski identificou corretamente os candidatos a tamanho de chave, e o Índice de Coincidência selecionou o candidato correto entre eles, levando à recuperação exata da chave e do plaintext original.

Durante os testes, no entanto, identificou-se uma limitação prática relevante ao aplicar o ataque a criptogramas mais curtos (na ordem de 190 a 210 letras, com chaves de 10 e 14 caracteres). Em alguns desses casos, o Exame de Kasiski não encontrou nenhuma sequência de três letras repetida no criptograma, resultado esperado, já que a ocorrência de repetições depende da coincidência de o mesmo trecho do plaintext se repetir alinhado a uma distância múltipla do tamanho da chave, o que é estatisticamente raro em textos curtos. Nos casos em que o Kasiski retornou candidatos, observou-se ainda que a estimativa via IC podia convergir para um divisor ou múltiplo do tamanho real da chave, em vez do valor correto, devido ao ruído estatístico introduzido por colunas com poucas amostras.

A Tabela I resume a relação observada entre o volume de texto disponível (expresso em letras por coluna, isto é, comprimento total do criptograma dividido pelo tamanho da chave) e a corretude da estimativa.

Tabela I: Relação entre letras por coluna e corretude da estimativa

Letras totais	Letras/coluna	Estimativa correta?
207 (chave=14)	14,8	Não
193 (chave=10)	19,3	Não
421 (chave=14)	30,1	Não
834 (chave=10)	83,4	Sim
835 (chave=20)	41,8	Sim
1017 (chave=20)	50,9	Sim
1263 (chave=14)	90,2	Sim
4210 (chave=14)	300,7	Sim

Os dados indicam que, para esta implementação, é necessário um volume mínimo de aproximadamente 80 a 90 letras por coluna para que a estimativa via IC convirja de forma estável para o tamanho correto da chave. Abaixo desse limiar, a estimativa fica sujeita a ruído amostral e pode indicar candidatos incorretos, tipicamente divisores ou múltiplos do valor real, já que colunas agrupadas segundo um divisor do tamanho verdadeiro da chave ainda preservam parte da estrutura estatística da língua. Esse comportamento é consistente com a literatura clássica de criptoanálise, que recomenda amostras de texto substancialmente maiores que o período da chave para garantir confiabilidade estatística no Teste de Friedman.

Como consequência prática, os criptogramas de teste mais curtos foram substituídos por amostras de maior volume (mantendo o mesmo idioma e a mesma chave), sem qualquer alteração nos algoritmos de Kasiski ou de Índice de Coincidência; a limitação identificada é inerente ao método estatístico e ao volume de dados disponível, não à implementação.

IV. CONCLUSÕES

Este trabalho implementou com sucesso a cifração e decifração da cifra de Vigenère, bem como um ataque de criptoanálise completo baseado em Exame de Kasiski, Índice de Coincidência e análise de frequência de letras. O ataque recuperou corretamente a chave e o plaintext original em todos os cenários com volume de texto adequado, em português e em inglês. Identificou-se, contudo, que a confiabilidade do ataque estatístico depende diretamente da razão entre o comprimento do criptograma e o tamanho da chave: abaixo de um limiar de aproximadamente 80 letras por coluna, a estimativa do tamanho da chave torna-se instável, favorecendo divisores ou múltiplos do valor correto. Esse resultado reforça, na prática, a discussão teórica da Seção II sobre os limites do ataque estatístico frente a criptogramas curtos, e ilustra por que, nesse regime, nem a força bruta nem a análise estatística isoladamente oferecem garantias satisfatórias de sucesso.

REFERÊNCIAS

- [1] C. E. Shannon, "Communication theory of secrecy systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949.

- [2] C.-K. Shene, "Kasiski's method," <https://pages.mtu.edu/~shene/NSF-4/Tutorial/VIG/Vig-Kasiski.html>, Michigan Technological University. Acesso em: ago. 2026.
- [3] A. L. Hananto, A. Solehudin, A. S. Y. Irawan, and B. S. Priyatna, "Analyzing the kasiski method against vigenere cipher," *International Journal of Computer Techniques*, vol. 6, no. 6, 2019, arXiv:1912.04519.
- [4] C.-K. Shene, "Index of coincidence," <https://pages.mtu.edu/~shene/NSF-4/Tutorial/VIG/Vig-IOC.html>, Michigan Technological University. Acesso em: ago. 2026.
- [5] C. Christensen, "Cryptanalysis of the vigenère cipher: The friedman test," <https://websites.nku.edu/~christensen/1402\Friedman\%20test\%202.pdf>, Northern Kentucky University. Acesso em: ago. 2026.
- [6] "Frequência de letras," https://pt.wikipedia.org/wiki/Frequ%C3%Aancia_de_letras, wikipédia. Acesso em: ago. 2026.